

Project Report

Protecting the ALU Logo on Ethereum

Abstract

This project protects the ALU official logo by registering its exact file fingerprint on an Ethereum blockchain and by tokenising ownership of that logo into fungible shares. The result is a system that proves the logo's authenticity, records when it was registered, and models shared ownership using two complementary smart contracts.

Introduction

ALU's official logo is an important digital brand asset. If a copied or modified version of the logo is circulated without permission, it can be difficult to prove which file is the original one unless there is a trustworthy record attached to the exact file contents. A private database can store that information, but the database owner can change records later, and outside parties must simply trust the university's internal system. Copyright registration is useful legally, but it does not give a fast technical way to compare one digital file against another.

Blockchain improves this by storing an immutable record of the logo's SHA-256 hash. Because the hash is produced from the real image file, anyone can hash another copy of the logo and compare the result to the value stored on-chain. If the values match, the file is authentic. If the values differ, even by one pixel, the file has been changed.

Question 1: Exploring a Cryptocurrency Exchange Platform

For the exchange-platform section, Coinbase was used as the reference platform because it is one of the best-known cryptocurrency exchanges and clearly separates wallets, funding methods, and order-entry tools. During onboarding, the platform typically asks for basic identity information such as legal name, date of birth, residential address, phone number, email address, and a government-issued identity document. This verification exists because exchanges must follow Know Your Customer and Anti-Money Laundering rules. In the United States, that compliance is tied to Bank Secrecy Act and customer-identification obligations used to reduce fraud, sanctions evasion, and money laundering.

The dashboard is designed to show balances, watchlists, asset prices, and recent account activity in one place. The wallet area focuses on custody and transfers, while the trading area is where orders are placed. Three widely listed assets are Bitcoin, Ethereum, and Solana. Bitcoin is primarily used as a scarce digital store of value and peer-to-peer monetary asset. Ethereum is different because it supports smart contracts and decentralised applications, so it is both a cryptocurrency and a programmable platform. Solana is known for high throughput and lower transaction costs, which makes it attractive for fast-moving applications and consumer-scale blockchain activity.

To add a payment method, a user normally goes to the payments or funding section and links a bank account or debit card. A bank account usually requires the account holder name, bank selection or routing details, and an account-confirmation step. A debit card normally requires the card number, expiry date, CVV, billing address, and a verification step. A debit card is usually faster for instant purchases, while a bank transfer is usually cheaper for larger or less urgent transactions.

For the purchase simulation, Bitcoin was selected. On June 24, 2026, Coinbase's public spot endpoint returned a BTC price of \$59,811.805 per BTC. A market order buys immediately at the best available market price, while a limit order only executes if the market reaches the exact price set by the buyer. A market order is better when execution speed matters; a limit order is better when price control matters more than certainty. For a simple classroom example, a market order is the more realistic choice because it matches what a first-time user would most likely use. Ignoring variable trading fees for a moment, $\$100 / 59,811.805$ is about 0.001672 BTC. After fees,

the amount received would be slightly lower. In practice, the exact final quantity depends on the spread and the fee tier applied at the moment of purchase.

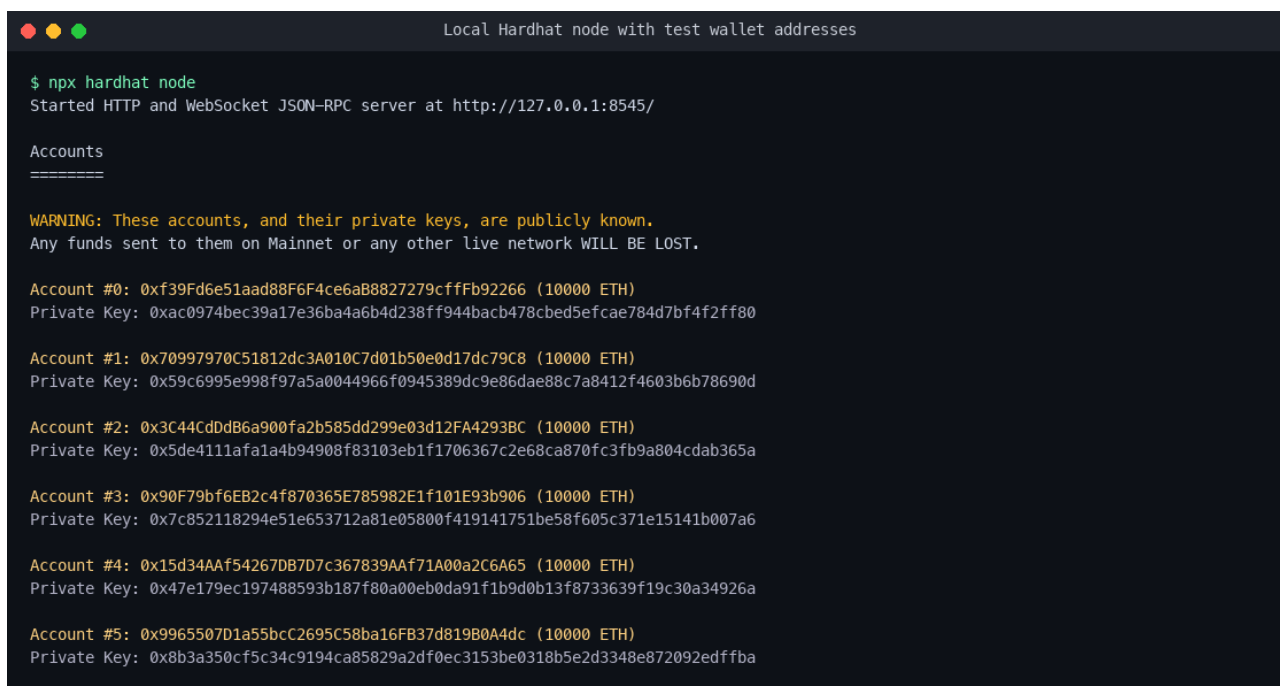
The transaction history section of an exchange records details such as the date, asset, quantity, total value, fees, status, and whether the transaction was a buy, sell, send, or receive operation. When a transaction is settled on-chain, it may also include a transaction hash. A transaction hash is the unique identifier of a blockchain transaction. It matters because it lets anyone independently confirm that the transfer was broadcast and recorded on the blockchain.

Two common security features on cryptocurrency exchanges are two-factor authentication and withdrawal protection. Two-factor authentication requires a second verification step, such as an authenticator code, even if the password is stolen. Withdrawal protections such as address confirmation, new-device checks, or allowlisting reduce the chance that an attacker can immediately move funds out of the account. These controls matter because exchange accounts combine financial access with irreversible digital-asset transfers.

Blockchain Setup

The development environment for this project uses Node.js, Hardhat, and OpenZeppelin Contracts. Node.js provides the JavaScript runtime needed for Hardhat scripts and tests. Hardhat compiles Solidity code, runs a local blockchain, executes tests, and deploys contracts. OpenZeppelin provides audited implementations of ERC-721, ERC-20, and ownership utilities so the project can build on trusted standards instead of rewriting them from scratch.

A local blockchain is a private Ethereum-compatible network that runs on the developer's own machine. Developers use it during development because it is fast, free, and safe to reset. Transactions confirm instantly, test accounts come pre-funded with fake ETH, and mistakes do not cost real money. A wallet address is the public account identifier used to send transactions, receive assets, and own tokens on the blockchain.



```
Local Hardhat node with test wallet addresses

$ npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
=====

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7Dc367839Aaf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bc2695C58ba16FB37d819B0A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba
```

Figure 1. Local Hardhat node running with pre-funded test accounts.

Smart Contract Explanation

ALUAssetRegistry

ALUAssetRegistry is the contract that protects the logo itself. It extends the ERC-721 standard because the ALU logo is a unique asset rather than a set of interchangeable units. The contract stores a struct called AssetMetadata with the asset name, file type, SHA-256 content hash, registering wallet address, and registration timestamp.

The contract also uses two mappings. One maps each token ID to its metadata so anyone can retrieve the stored information later. The second maps each content hash to a boolean value so duplicate registration attempts can be blocked.

registerAsset()

registerAsset() accepts an asset name, file type, and bytes32 content hash. It first checks whether the same hash was used before. If so, it reverts with 'Asset hash already registered'. If the hash is new, the function increments the token counter, mints a new NFT to the caller with `_safeMint()`, stores the metadata, marks the hash as used, emits AssetRegistered, and returns the new token ID.

verifyLogoIntegrity()

verifyLogoIntegrity() is a view function that accepts a token ID and a hash provided by the caller. It compares the supplied hash to the stored hash for that token. If they match, it returns true and the message 'Logo is authentic.' If they do not match, it returns false and the message 'Warning: logo does not match.' Because it only reads blockchain data, it does not require gas in a local call context.

getAsset()

getAsset() returns the full stored metadata for a registered token. This allows public inspection of the asset's identity, its file type, who registered it, and when registration occurred.

Real ALU Logo Hash

The official ALU logo file used in this project is alu-logo.png, downloaded from ALU's public website. Its SHA-256 hash was generated locally with the scripts/hash-logo.js helper.

SHA-256: 727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560

bytes32: 0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560

A terminal window titled "SHA-256 of alu-logo.png" showing the command execution and output. The command is `$ npm run hash:logo`. The output shows the SHA-256 hash and the corresponding bytes32 value.

```
SHA-256: 727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560
bytes32: 0x727b0a610a5e22c083efc8b467c1580cad4b2626d8a6171d686a99d6a220a560
```

Figure 2. SHA-256 fingerprint of the alu-logo.png file.

SHA-256 is useful because it acts like a fingerprint for the file. If even one pixel of the logo changes, the resulting hash changes completely. That is what makes it reliable for integrity checking.

Blockchain data is hard to change because transactions are recorded in blocks that are cryptographically linked and replicated across the network. Once accepted, altering past data would require rewriting chain history and re-establishing consensus, which is exactly what gives the stored record its practical immutability.

Tokenisation

After registering the logo as a unique NFT, the project tokenises ownership using a second contract called ALULogoToken. This contract extends ERC-20 because ownership shares are fungible: one ALUT token is interchangeable with another ALUT token in a way that an NFT is not.

The contract creates a token named 'ALU Logo Token' with symbol ALUT and a fixed total supply of 1,000,000 tokens. On deployment, the full supply is minted to the owner address provided to the constructor. That means the original owner starts with 100% of the tokenised share supply.

distributeShares() is restricted with onlyOwner, so only the contract owner can distribute token shares to other addresses. This models how ALU could allocate ownership shares to authorised parties such as administration, brand-management teams, or internal stakeholders. ownershipPercentage() calculates a wallet's share of the full 1,000,000-token supply and returns it as a whole-number percentage.

ERC-721 is the correct standard for the logo registration because the logo itself is a single, unique asset. ERC-20

is the correct standard for the share model because ownership units must be fungible and transferable in equal amounts. If a faculty member holds 100,000 ALUT out of 1,000,000, that means the faculty member holds 10% of the tokenised ownership supply.

A terminal window with a dark background and light green text. The title bar at the top says "Deploy script: both contracts + logo registered". The command entered is "\$ ALU_LOGO_HASH=0x727b0a...a220a560 npx hardhat run scripts/deploy.js". The output shows the deployment details for the ALUAssetRegistry, ALU logo, and ALULogoToken contracts, including their respective addresses and the token ID (1).

```
Deploy script: both contracts + logo registered

$ ALU_LOGO_HASH=0x727b0a...a220a560 npx hardhat run scripts/deploy.js

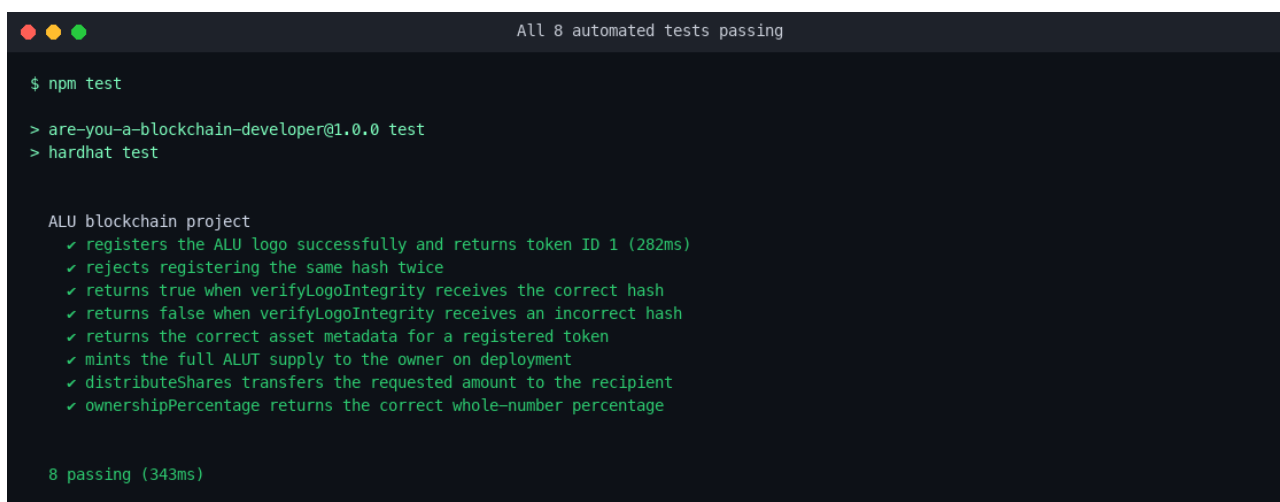
Deploying with account: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfFb92266
ALUAssetRegistry deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
ALU logo registered with token ID: 1
ALULogoToken deployed to: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
```

Figure 3. Deployment script deploying both contracts and registering the ALU logo.

Test Results

The project includes eight automated tests. All eight pass on the local Hardhat network.

- The ALU logo registers successfully and returns token ID 1.
- A second attempt to register the same hash is rejected.
- verifyLogoIntegrity() returns true for the correct hash.
- verifyLogoIntegrity() returns false for an incorrect hash.
- getAsset() returns the correct asset metadata.
- ALULogoToken mints the full 1,000,000 supply to the owner on deployment.
- distributeShares() transfers the requested amount to a recipient.
- ownershipPercentage() returns the correct whole-number percentage.

A terminal window with a dark background and light green text. The title bar at the top says "All 8 automated tests passing". The command entered is "\$ npm test". The output shows the results of the tests, including the ALU blockchain project name and a list of eight tests that all passed. The total time for the tests is 343ms.

```
All 8 automated tests passing

$ npm test

> are-you-a-blockchain-developer@1.0.0 test
> hardhat test

ALU blockchain project
  ✓ registers the ALU logo successfully and returns token ID 1 (282ms)
  ✓ rejects registering the same hash twice
  ✓ returns true when verifyLogoIntegrity receives the correct hash
  ✓ returns false when verifyLogoIntegrity receives an incorrect hash
  ✓ returns the correct asset metadata for a registered token
  ✓ mints the full ALUT supply to the owner on deployment
  ✓ distributeShares transfers the requested amount to the recipient
  ✓ ownershipPercentage returns the correct whole-number percentage

8 passing (343ms)
```

Figure 4. Screenshot of all eight automated tests passing.

Conclusion

This project demonstrates how blockchain can protect a digital asset and how token standards solve different parts of the ownership problem. The ERC-721 contract creates a permanent authenticity record for the ALU logo, while the ERC-20 contract creates a practical representation of shared ownership. If more time were available, the next step would be to build the dApp interface mentioned in the assignment so users could hash files, verify authenticity, and distribute shares through a web application.

Links

GitHub repository:

<https://github.com/tejirijesse/are-you-a-blockchain-developer>

Demo video:

<link to be added>